

#08 Set, Map

김성수

4thrunyul@gmail.com

0. 목차



1. Beyond Searching

- RB Tree VS Hash Table

2. Set

3. Map

1. Beyond Searching

1. Beyond Searching

Red-Black Tree

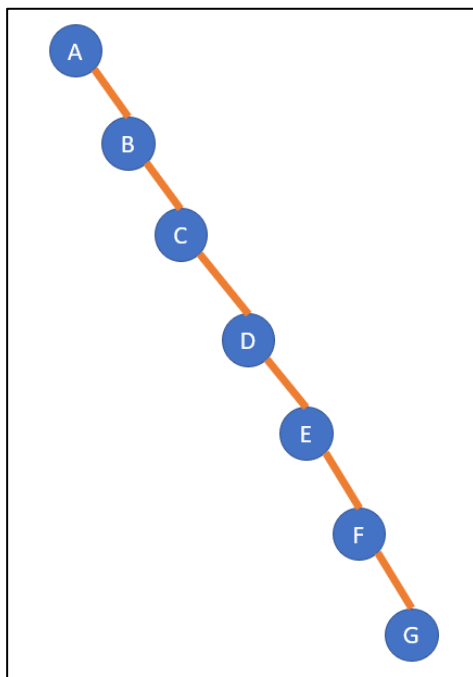
- 정렬된 데이터 셋에서 최선의 탐색 방법 => 이분 탐색, $O(\log n)$
- 탐색은 그렇다 치고, 삽입, 수정은?
- 기존 vector, array에서는 삽입, 수정을 하기 위해서는 $O(n)$
- 아래 예제에서 1을 삽입하되, 정렬을 유지하기 위해서는 $O(n)$ 만큼 시간이 걸림

$$v = \{7, 64, 124, 156, 1055, 7594\}$$

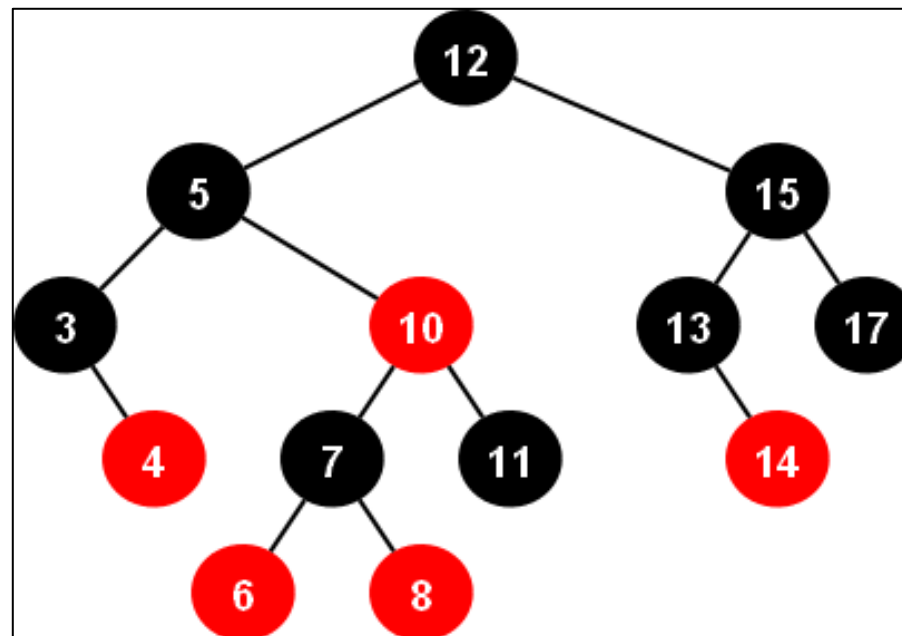
1. Beyond Searching

Red-Black Tree

- 자가 균형 이진 탐색 트리
- 탐색, 삽입, 수정 $O(\log n)$



▲ 편향된 이진 탐색 트리



▲ RB Tree

1. Beyond Searching

Hashing(해싱)

- 선형 탐색 : $O(n)$
- 이분 탐색
 - 정렬이 되어있는 경우 : $O(\log n)$
 - 정렬이 안 되어있는 경우 : $O(n \log n)$
- 더 빠르게 탐색하는 방법이 있을까?

$$v = \{54, 7, 59, 2952, 23, 0, 6\}$$

1. Beyond Searching

Hashing(해싱)

- 데이터를 해시 함수를 이용해 고정된 값(해시값)으로 변환하고, 이를 통해 데이터를 빠르게 저장하거나 탐색하는 기법
- 연산 한 번을 통해 해당 값 존재 여부를 확인할 수 있음 => 평균 $O(1)$
- 아래 예제에서 해시 함수는 $f(n) = n \% 10$ 이라고 설정

$v = \{54, 7, 59, 2952, 23, 0, 6\}$

hash table = $\{4, 7, 9, 2, 3, 0, 6\}$

1. Beyond Searching

Hashing(해싱)

- 해시 충돌(Hash Collision) 가능성을 배제할 수 없음
- 충돌을 막기 위해 보통 해시 함수에서 나머지 연산은 소수(Prime Number)를 사용
- 충돌이 일어나면 Chainig, Open Addressing 방법을 통해 제어 => 최악 $O(n)$
- 아래 예제에서 해시 함수는 $f(n) = n \% 10$ 이라고 설정
- 여기서 33을 해싱하면 기존 23의 해시값과 동일하여 충돌이 발생

$v = \{54, 7, 59, 2952, 23, 0, 6\}$

hash table = $\{4, 7, 9, 2, 3, 0, 6\}$

1. Beyond Searching



RB Tree vs Hash Table

- 오늘 배울 set과 map은 기본적으로 RB Tree 형태로 이루어져 있음
- 하지만 unordered_set, unordered_map은 Hash Table 형태로 이루어져 있음
- RB Tree
 - 데이터의 순서가 중요한 경우
 - 일관된 성능이 필요할 때(모든 연산이 $O(\log N)$ 이 보장됨)
- Hash Table
 - 데이터의 양이 방대하고, 단순 탐색 성능이 가장 중요한 경우
 - 메모리 공간보다 시간이 중요할 때

2. Set

2. Set



Set(셋, 세트)

- 데이터를 중복 없이 저장하는 자료구조
- 내부적으로는 레드 블랙 트리(Red-Black Tree)로 구성되어 삽입과 동시에 자동으로 오름차순 정렬됨.

2. Set

```

1  #include <iostream>
2  #include <set>
3
4  using namespace std;
5
6  int main() {
7      set<int> s = {5, 2, 3, 2, 6};
8
9      s.insert(7); // {2, 3, 5, 6, 7}
10     s.erase(2); // {3, 5, 6, 7}
11
12     cout << s.erase(6) << "\n"; // {3, 5, 7}
13     cout << s.erase(2) << "\n"; // {3, 5, 7}
14     cout << s.size();
15
16     return 0;
17 }

```

Input:

Copy

Expected Output:

Copy

1
0
3

set<자료형> 변수명 | <set>

- 셋을 선언하기 위해서는 <set> 라이브러리 필요
- 자료형에는 거의 모든 자료형을 사용할 수도 있으며, 구조체를 선언하여 자료형으로 넣을 수도 있음

set.insert(val) | <set> | $O(\log n)$

- 셋에 새로운 데이터를 추가하여 저장하며, 자동으로 오름차순 정렬을 유지함
- 셋에 이미 존재하는 원소를 삽입할 경우 무시됨

set.erase(val) | <set> | $O(\log n)$

- 셋에서 특정 값을 가진 데이터를 찾아 제거하며, 자동으로 오름차순 정렬을 유지함.
- 단순히 set.erase()을 수행시킬 수도 있고, 이를 출력할 수도 있음.
- 삭제 성공 시 1, 실패 시 0을 반환함.

set.size() | <set> | $O(1)$

- 셋에 현재 저장되어 있는 데이터의 총 개수를 확인하는 함수
- 내부 컨테이너에 요소의 수를 저장하고 있어서 $O(1)$ 의 시간 복잡도를 가짐

2. Set

```
1  #include <iostream>
2  #include <set>
3
4  using namespace std;
5
6  int main() {
7      set<int> s = {5, 2, 3, 2, 6};
8
9      if(s.find(2) != s.end()) s.erase(2);
10     // {3, 5, 6}
11
12     cout << s.count(2) << "\n";
13     cout << s.count(3) << "\n";
14
15     auto it = s.lower_bound(5);
16
17     if(it != s.end()) cout << "Found\n";
18     else cout << "Not Found\n";
19
20     it = s.upper_bound(6);
21
22     if(it != s.end()) cout << *it;
23     else cout << "Not Found";
24
25     return 0;
26 }
```

set.find(val) | <set> | $O(\log n)$

- 셋에서 특정 값을 가진 데이터를 찾는 함수
- 데이터를 찾았을 경우 해당 요소의 반복자를 반환하며, 찾지 못했을 경우 set.end()를 반환

set.count(val) | <set> | $O(\log n)$

- 셋에서 특정 값을 가진 데이터의 개수를 확인하는 함수
- 셋은 중복 원소를 허용하지 않으므로 데이터가 존재하면 1, 없으면 0을 반환

set.lower_bound(val), set.upper_bound(val) | <set> | $O(\log n)$

- 셋의 원소들 중 인자로 전달된 값 이상의/초과하는 첫 번째 원소의 반복자를 반환함.
- 데이터를 찾지 못했을 경우 set.end()를 반환
- unordered_set에서는 사용 불가능

set.clear() | $O(n)$

- 셋의 내부를 초기화하는 함수

```
Input:                                     Copy
Expected Output:                           Copy
0
1
Found
6
```

2. Set



```
1  #include <iostream>
2  #include <set>
3
4  using namespace std;
5
6  int main() {
7      set<int> s = {40, 10, 30, 20, 50};
8
9      s.insert(25);
10
11     for (const int& n : s) {
12         cout << n << " ";
13     }
14
15     return 0;
16 }
```

범위 기반 for문

- vector, string과 마찬가지로 set에도 범위 기반 for문을 사용하여 요소를 순회할 수 있음

const

- 데이터를 읽기 전용으로 설정하여 프로그램 실행 중 값이 수정되는 것을 방지
 - 원소가 수정되면 정렬 순서가 망가지기 때문에, 애초에 원소를 읽기 전용으로 관리

&(참조자)

- 기존 변수에 새로운 이름을 붙여 메모리 주소를 직접 공유하는 방식
- 데이터를 복사하지 않고 원본에 접근하므로 메모리 사용량을 줄이고 성능을 향상시킴
 - 원소가 수정되면 정렬 순서가 망가지기 때문에, 일반 참조자 사용을 막아 둠
 - 참조자를 사용하려면 명시적으로 const를 함께 써주어야 함

Input:

Copy

Expected Output:

Copy

10 20 25 30 40 50

2. Set: 실습



실습

- 10815 숫자 카드
 - 코드
- 7785 회사에 있는 사람
 - 코드

3. Map

3. Map



Map(맵)

- 키(Key)와 값(Value)이 한 쌍을 이루어 데이터를 저장하는 자료구조
- 중복된 키를 허용하지 않음.
- 내부적으로는 레드 블랙 트리(Red-Black Tree)로 구성되어 삽입과 동시에 키(Key)를 기준으로 자동 오름차순 정렬됨.

3. Map

```

1  #include <iostream>
2  #include <map>
3  #include <string>
4
5  using namespace std;
6
7  int main() {
8      map<int, string> m1 = {
9          {1, "Banana"},
10         {5, "Apple"},
11         {121, "WaterMelon"}
12     };
13
14     m1.insert({7, "Melon"});
15     m1[90] = "Pear";
16
17     cout << m1[7] << "\n";
18     cout << m1[90] << "\n";
19
20     map<int, int> m2 = {
21         {1, 10},
22         {3, 5}
23     };
24
25     m2[2]++;
26     m2[2] += 2;
27
28     cout << m2[2] << "\n";
29
30     return 0;
31 }

```

map<key 자료형, value 자료형> 변수명 | <map>

- 맵을 선언하기 위해서는 <map> 라이브러리 필요
- 자료형에는 거의 모든 자료형을 사용할 수도 있으며, 구조체를 선언하여 자료형으로 넣을 수도 있음

m.insert({key, value}) | <map> | $O(\log n)$

- 맵에 새로운 데이터를 추가하여 저장하며, key를 기준으로 자동으로 오름차순 정렬을 유지함
- 맵에 이미 존재하는 key를 삽입할 경우 무시됨

m.erase(key) | <map> | $O(\log n)$

- 맵에서 특정 key를 가진 데이터를 찾아 제거하며, 자동으로 오름차순 정렬을 유지함.
- 단순히 m.erase()을 수행시킬 수도 있고, 이를 출력할 수도 있음.
- 삭제 성공 시 1, 실패 시 0을 반환함.

m.size() | <map> | $O(1)$

- 맵에 현재 저장되어 있는 데이터의 총 개수를 확인하는 함수
- 내부 컨테이너에 요소의 수를 저장하고 있어서 $O(1)$ 의 시간 복잡도를 가짐

```

Input:                                     Copy
-----
Expected Output:                           Copy
Melon
Pear
3

```

3. Map

```
1  #include <iostream>
2  #include <map>
3  #include <string>
4
5  using namespace std;
6
7  int main() {
8      map<int, string> m = {
9          {1, "Banana"},
10         {5, "Apple"},
11         {121, "WaterMelon"},
12         {84, "Pear"},
13         {74, "Mango"}
14     };
15
16     if(m.find(5) != m.end()) cout << m[5] << "\n";
17     else cout << "Can't find\n";
18
19     if(m.find(3) != m.end()) cout << m[3] << "\n";
20     else cout << "Can't find\n";
21
22     if(m.count(121)) cout << m[121] << "\n";
23
24     // key값이 50이상 100이하인 모든 key, value 출력
25     auto it1 = m.lower_bound(5);
26     auto it2 = m.upper_bound(100);
27
28     for(auto it = it1; it != it2; ++it) {
29         cout << (*it).first << " : " << (*it).second << "\n";
30     }
31
32     return 0;
33 }
```

`m.find(key) | <map> |  $O(\log n)$`

- 맵에서 특정 key를 가진 데이터를 찾는 함수
- 데이터를 찾았을 경우 해당 요소의 반복자를 반환하며, 찾지 못했을 경우 `set.end()`를 반환

`m.count(key) | <map> |  $O(\log n)$`

- 맵에서 특정 key를 가진 데이터의 개수를 확인하는 함수
- 맵은 중복 key를 허용하지 않으므로 데이터가 존재하면 1, 없으면 0을 반환

`m.lower_bound(key), m.upper_bound(key) | <map> |  $O(\log n)$`

- 맵의 원소들 중 인자로 전달된 값 이상의/초과하는 첫 번째 key의 반복자를 반환함.
- 데이터를 찾지 못했을 경우 `m.end()`를 반환
- `unordered_map`에서는 사용 불가능

`map.clear() |  $O(n)$`

- 맵의 내부를 초기화하는 함수

```
Input:                                     Copy
Expected Output:                           Copy
Apple
Can't find
WaterMelon
5 : Apple
74 : Mango
84 : Pear
```

3. Map

```

1  #include <iostream>
2  #include <map>
3  #include <string>
4
5  using namespace std;
6
7  int main() {
8      map<int, string> m = {
9          {1, "Banana"},
10         {84, "Pear"},
11         {74, "Mango"}
12     };
13
14     // pair 컨테이너 이용
15     for(const auto& kv : m) {
16         cout << kv.first << " : " << kv.second << "\n";
17     }
18
19     // 구조 분해 할당 이용
20     for(const auto& [key, value] : m) {
21         cout << key << " : " << value << "\n";
22     }
23
24     return 0;
25 }

```

Input: [Copy](#)

Expected Output: [Copy](#)

```

1 : Banana
74 : Mango
84 : Pear
1 : Banana
74 : Mango
84 : Pear

```

범위 기반 for문

- vector, string과 마찬가지로 map에도 범위 기반 for문을 사용하여 요소를 순회할 수 있음

const

- 데이터를 읽기 전용으로 설정하여 프로그램 실행 중 값이 수정되는 것을 방지
 - 원소가 수정되면 정렬 순서가 망가지기 때문에, 애초에 원소를 읽기 전용으로 관리

&(참조자)

- 기존 변수에 새로운 이름을 붙여 메모리 주소를 직접 공유하는 방식
- 데이터를 복사하지 않고 원본에 접근하므로 메모리 사용량을 줄이고 성능을 향상시킴
 - 원소가 수정되면 정렬 순서가 망가지기 때문에, 일반 참조자 사용을 막아 둠
 - 참조자를 사용하려면 명시적으로 const를 함께 써주어야 함

3. Map

일대 다 매핑

- 하나의 key에 여러 개의 value가 매핑되는 구조

map<key, vector<value>>

- value가 vector 안에 저장되는 구조이기 때문에 key를 통한 Lookup*이후 벡터에 접근하여 인덱싱할 수 있음

```

1  #include <iostream>
2  #include <map>
3  #include <vector>
4  #include <string>
5
6  using namespace std;
7
8  int main() {
9      map<int, vector<string>> m = {
10         {0, {"one", "two", "three"}},
11         {1, {"spring", "summer"}}
12     };
13
14     m[0].push_back("four");
15     m[1].push_back("autumn");
16
17     for(auto it : m) {
18         cout << it.first << " : ";
19
20         for(int i = 0; i < it.second.size(); i++) {
21             cout << it.second[i] << " ";
22         }
23
24         cout << "\n";
25     }
26
27     return 0;
28 }

```

Input:

Copy

Expected Output:

Copy

```

0 : one two three four
1 : spring summer autumn

```

* Lookup(조회, 검색) : Key를 통해 Value를 찾아내는 행위

3. Map: 실습



실습

- [10816 숫자 카드 2](#)
 - [코드](#)
- [17219 비밀번호 찾기](#)
 - [코드](#)